

Rest and Spread Operators

Table of Contents

1. Introduction
2. Spread Operator (...)
3. Rest Operator (...)
4. ES6 Modules with Rest/Spread
5. Practical Examples
6. Best Practices

Introduction

The Rest and Spread operators in JavaScript use the same syntax (`...`) but serve opposite purposes:

- **Spread Operator:** Expands/spreads elements from arrays, objects, or iterables
- **Rest Operator:** Collects/gathers multiple elements into a single array or object

Spread Operator (...)

1. Object Spread

The spread operator allows you to copy properties from one object to another or merge multiple objects.

```
// Basic object spread
const user = {
  username: "tinku",
  age: 22,
  city: "Noida",
  pin: 201301
}

// Creating a new object with spread
const updatedUser = {
  ...user,
  age: 23,           // Override existing property
  phone: "9876543210" // Add new property
}

console.log(updatedUser);
// Output: { username: "tinku", age: 23, city: "Noida", pin: 201301, phone: "9876543210" }
```

2. Merging Multiple Objects

```
// From your examples
const personal = {
  maritalStatus: "Unmarried",
  gender: "male"
}

const company = {
  compName: "Clothzy Fashion Tech",
  Address: "Sector-142 Noida",
  salary: "3.4 to 5.4 LPA",
  Desg: "Mern Stack Developer"
}

const username = {
  name: "Avinash",
  skills: ["react", "Node", "Express", "Mongo"],
  ...personal,    // Spread personal object
  ...company      // Spread company object
}

console.log(username);
// Output: Complete merged object with all properties
```

3. Array Spread

```
const shop = {
  biscuits: ["oreo", "parleg", "Jim-jam"],
  coldDrinks: ["mazza", "campa", "sprite", "coke"]
}

// Spreading arrays
const allItems = [...shop.biscuits, ...shop.coldDrinks];
console.log(allItems);
// Output: ["oreo", "parleg", "Jim-jam", "mazza", "campa", "sprite", "coke"]

// Adding new items while spreading
const expandedBiscuits = [...shop.biscuits, "good-day", "bourbon"];
console.log(expandedBiscuits);
// Output: ["oreo", "parleg", "Jim-jam", "good-day", "bourbon"]
```

4. Function Arguments Spread

```
function calculateSum(a, b, c) {
  return a + b + c;
}

const numbers = [10, 20, 30];
const result = calculateSum(...numbers); // Spread array as arguments
console.log(result); // Output: 60
```

Rest Operator (...)

1. Object Rest (Destructuring)

The rest operator collects remaining properties into a new object.

```
// From your example
const user = {
  username: "tinku",
  age: 22,
  city: "Noida",
  pin: 201301
}

const {username, ...obj} = user; // Rest collects remaining properties

console.log(username); // Output: "tinku"
console.log(obj);      // Output: { age: 22, city: "Noida", pin: 201301 }
```

2. Array Rest (Destructuring)

```
// From your examples
const shop = {
  biscuits: ["oreo", "parleg", "Jim-jam"],
  coldDrinks: ["mazza", "campa", "sprite", "coke"]
}

// Array destructuring with rest
const {biscuits: [first, ...item1], coldDrinks: [firstDrink, ...item2]} = shop;

console.log(first);      // Output: "oreo"
console.log(item1);      // Output: ["parleg", "Jim-jam"]
console.log(firstDrink); // Output: "mazza"
console.log(item2);      // Output: ["campa", "sprite", "coke"]
```

3. Array of Objects Rest

```
// From your example
const employee = [
  {empName: "Avinash", desg: "Software developer"},
  {empName: "Ranjan", desg: "Frontend developer"}
]

const [e1, ...res] = employee; // Rest collects remaining array elements

console.log(e1); // Output: {empName: "Avinash", desg: "Software developer"}
console.log(res); // Output: [{empName: "Ranjan", desg: "Frontend developer"}]
```

4. Function Parameters Rest

```
function processEmployees(manager, ...employees) {
  console.log("Manager:", manager);
  console.log("Employees:", employees);
}

processEmployees("John", "Alice", "Bob", "Charlie");
// Output:
// Manager: John
// Employees: ["Alice", "Bob", "Charlie"]
```

ES6 Modules with Rest/Spread

Export Statements Rules

```
// main.js
const user = {
  username: "tinku",
  age: 22,
  city: "Noida",
  pin: 201301
}

const employee = [
  {empName: "Avinash", desg: "Software developer"},
  {empName: "Ranjan", desg: "Frontend developer"}
]

// Export statement must be the last statement of the code
export {
  user,
  employee
};
```

Default Exports

```
// spread.js
const company = {
  compName: "Clothzy Fashion Tech",
  Address: "Sector-142 Noida",
  salary: "3.4 to 5.4 LPA",
  Desg: "Mern Stack Developer"
}

// 'default' is only used when one object is to be exported
export default company;

// sum.js
const personal = {
  maritalStatus: "Unmarried",
  gender: "male"
}

export default personal;
```

Import and Usage

```
// script.js
import {user, employee} from "./main.js";
import company from "./spread.js";    // Default import
import personal from "./sum.js";      // Default import

// Using rest with imported data
const {username, ...userRest} = user;
const [firstEmployee, ...otherEmployees] = employee;

// Using spread with imported data
const completeProfile = {
  name: "Avinash",
  skills: ["react", "Node", "Express", "Mongo"],
  ...personal,    // Spread imported object
  ...company      // Spread imported object
}

console.log(completeProfile);
```

Practical Examples

1. Cloning Objects and Arrays

```
// Shallow clone of object
const originalUser = {name: "John", age: 25};
const clonedUser = {...originalUser};

// Shallow clone of array
const originalArray = [1, 2, 3, 4, 5];
const clonedArray = [...originalArray];
```

2. Removing Properties from Objects

```
const userWithPassword = {
  username: "tinku",
  password: "secret123",
  age: 22,
  city: "Noida"
}

// Remove password using rest
const {password, ...safeUser} = userWithPassword;
console.log(safeUser); // Object without password property
```

3. Function with Dynamic Parameters

```
function combineSkills(primarySkill, ...otherSkills) {
  return {
    primary: primarySkill,
    secondary: otherSkills
  };
}

const skillSet = combineSkills("JavaScript", "React", "Node.js", "MongoDB");
console.log(skillSet);
// Output: { primary: "JavaScript", secondary: ["React", "Node.js", "MongoDB"] }
```

4. Conditional Object Properties

```
const createEmployeeProfile = (name, designation, isManager = false) => {
  return {
    name,
    designation,
    ...(isManager && { managementLevel: "Senior" }), // Conditional spread
    skills: []
  };
}
```

Best Practices

1. Order Matters in Spread

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };

const merged1 = { ...obj1, ...obj2 }; // { a: 1, b: 3, c: 4 } - obj2.b overwrites obj1.b
const merged2 = { ...obj2, ...obj1 }; // { b: 2, c: 4, a: 1 } - obj1.b overwrites obj2.b
```

2. Shallow Copy Limitation

```
const original = {
  user: { name: "John" },
  skills: ["JS", "React"]
};

const copied = { ...original };
copied.user.name = "Jane"; // This will modify the original object too!

// For deep cloning, use other methods like JSON.parse(JSON.stringify()) or libraries
```

3. Performance Considerations

- Spread operator creates new objects/arrays, so use wisely in loops
- Rest operator is efficient for collecting remaining elements
- Prefer rest/spread over traditional methods for better readability

4. Export Guidelines

- Export statements should be at the end of the file
- Use `default` export only when exporting a single primary entity
- Named exports are preferred for multiple entities
- Import names should be descriptive and consistent

Summary

- **Spread (...):** Expands elements → `{...obj}`, `[...arr]`, `func(...args)`
- **Rest (...):** Collects elements → `{prop, ...rest}`, `[first, ...rest]`, `func(param, ...rest)`
- Both operators provide clean, readable syntax for common operations
- Essential for modern JavaScript development with ES6 modules
- Remember: same syntax, opposite purposes!